

**LAPORAN PROYEK SISTEM PERPUSTAKAAN DIGITAL
MENGUNAKAN JAVA**

Disusun untuk tugas mata kuliah Algoritma dan Struktur Data

Dosen Pengampu:

Taufik Ridwan M.T.



Disusun oleh:

Kelompok 6

Anfa' Annas 2510631250024

Link Github:

<https://github.com/takeacigs/Library-system-digital>

Muhammad Tammam Alkhairi 2510631250039

Link Github:

<https://github.com/TammamAlkhairi/ProjekUas>

Wulan Ramadhani 2510631250003

Link Github:

<https://github.com/2510631250003-glitch/ProjectWulanRamadhani>

**PROGRAM STUDI SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG**

2026

Daftar Isi

BAB 1 PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Tujuan.....	3
BAB 2 ANALISIS KEBUTUHAN.....	4
2.1 Analisis Fitur Sistem.....	4
2.2 Struktur Data.....	4
BAB 3 IMPLEMENTASI.....	6
3.1 Struktur Program.....	6
3.2 Implementasi CRUD.....	7
3.3 Implementasi Searching.....	8
3.4 Implementasi Sorting.....	9
3.5 Fitur Tambahan.....	10
BAB 4 ANALISIS KOMPLEKSITAS.....	11
4.1 Tabel Kompleksitas Waktu.....	11
4.2 Kompleksitas Ruang (Space Complexity).....	12
4.3 Penjelasan Detail.....	13
BAB 5 PENGUJIAN.....	14
5.1 Skenario 1 Tampil Data (Normal).....	14
5.2 Skenario 2 Tambah Data (Berhasil).....	15
5.3 Skenario 3 Tambah Data (ID Duplikat).....	15
5.4 Skenario 4 Tambah Data (Jumlah Negatif).....	16
5.5 Skenario 5 Edit Data.....	16
5.7 Skenario 7 Hapus Data (ID Tidak Ada).....	17
5.8 Skenario 8 Cari Nama (Linear Search).....	18
5.9 Skenario 9 Cari Nama (Tidak Ditemukan).....	18
5.10 Skenario 10 Cari ID (Binary Search).....	19
5.11 Skenario 11 Cari Kategori.....	19
5.12 Skenario 12 Sort ID (Ascending).....	20
5.13 Skenario 13 Sort Nama (A-Z).....	20
5.14 Skenario 14 Sort Jumlah Descending.....	21
5.15 Skenario 15 Statistik Data.....	21
5.16 Skenario 16 Simpan dan Load Data.....	22
5.17 Skenario 17 Status Sistem.....	23
BAB 6 KESIMPULAN.....	24
6.1 Ringkasan.....	24
6.2 Kelebihan Sistem.....	24
6.3 Kelemahan dan Saran Perbaikan.....	24

6.4 Penutup.....	25
------------------	----

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Perpustakaan merupakan sarana penting dalam mendukung proses pembelajaran dan penyediaan informasi. Pengelolaan data buku yang masih dilakukan secara

manual sering menimbulkan berbagai masalah seperti kesulitan pencarian data, kesalahan pencatatan, serta proses pengelolaan yang lambat. Oleh karena itu, dibutuhkan sebuah sistem yang mampu membantu pengelolaan data perpustakaan secara lebih efektif dan efisien.

Program Sistem Perpustakaan Digital dibuat menggunakan bahasa pemrograman Java dengan konsep array, searching, sorting, CRUD, serta pengolahan file. Sistem ini dapat digunakan untuk mengelola data buku seperti menambah, menampilkan, mengedit, menghapus, mencari, mengurutkan, hingga menyimpan data ke file.

1.2 Tujuan

Tujuan dari pembuatan program ini adalah:

1. Membuat sistem pengelolaan data perpustakaan berbasis Java.
2. Mengimplementasikan operasi CRUD pada data buku.
3. Mengimplementasikan algoritma searching dan sorting.
4. Memahami penggunaan array sebagai struktur data.
5. Mengimplementasikan penyimpanan dan pembacaan data menggunakan file.
6. Menganalisis kompleksitas algoritma yang digunakan.

BAB 2

ANALISIS KEBUTUHAN

2.1 Analisis Fitur Sistem

Program memiliki beberapa fitur utama, yaitu:

1. CRUD (Create, Read, Update, Delete)

- a. Tambah data buku
 - b. Menampilkan data buku
 - c. Mengedit data buku
 - d. Menghapus data buku
2. Searching
- a. Pencarian berdasarkan nama buku
 - b. Pencarian berdasarkan ID buku
 - c. Pencarian berdasarkan kategori
3. Sorting
- a. Pengurutan berdasarkan ID
 - b. Pengurutan berdasarkan nama buku
 - c. Pengurutan berdasarkan jumlah buku
4. Additional Features
- a. Statistik data perpustakaan
 - b. Menyimpan data ke file
 - c. Memuat data dari file
 - d. Menampilkan status sistem

2.2 Struktur Data

Program menggunakan struktur data array untuk menyimpan data buku.

Nama Array	Tipe Data	Fungsi
id[]	int	Menyimpan ID buku
nama[]	String	Menyimpan nama buku
kategori[]	String	Menyimpan kategori buku
jumlah[]	int	Menyimpan jumlah stok buku
jumlahData[]	int	Menyimpan jumlah data buku aktif
counterTambah	int	Menghitung jumlah tambah data
counterEdit	int	Menghitung jumlah edit data
counterHapus	int	Menghitung jumlah hapus data
statusTerakhir	String	Menyimpan status aktivitas terakhir sistem
input	Scanner	Menerima input dari pengguna

BAB 3

IMPLEMENTASI

3.1 Struktur Program

Program terdiri dari satu class SistemPerpus dengan metode-metode statis yang dikelompokkan sesuai fungsinya. Navigasi dilakukan melalui menu bertingkat menggunakan *do-while* dan *switch*.

main()

└─ menuCRUD()

- | └─ tambahData()
- | └─ tampilData()
- | └─ editData()
- | └─ hapusData()
- └─ menuSearching()
 - | └─ cariNama() → linearSearch pada array nama
 - | └─ cariID() → bubbleSortID() lalu binarySearch()
 - | └─ cariKategori() → linearSearch pada array kategori
- └─ menuSorting()
 - | └─ sortID() → bubbleSortID()
 - | └─ sortNama() → selectionSortNama()
 - | └─ sortJumlahDescending() → bubbleSortDesc()
- └─ menuFiturTambahan()
 - └─ tampilStatistik()
 - └─ simpanData()
 - └─ loadData()
 - └─ tampilStatusCounter()

3.2 Implementasi CRUD

tambahData() adalah Menambah buku baru ke array pada posisi jumlahData:

```
// Boundary handling jika kapasitas array penuh
if (jumlahData >= id.length) {
    System.out.println("Kapasitas penyimpanan penuh");
    return;
}
```

```
// Boundary handling ID duplikat
int idBaru = input.nextInt();

if (linearSearchID(idBaru) != -1) {
    System.out.println("ID sudah digunakan");
    return;
}
```

```
// Boundary handling jumlah negatif
int jumlahBuku = input.nextInt();

if (jumlahBuku < 0) {
    System.out.println("Jumlah buku tidak boleh negatif");
    return;
}

jumlah[jumlahData] = jumlahBuku;
jumlahData++;
```

hapusData() adalah menghapus buku dengan menggeser elemen ke kiri (shift left):

```
for (int i = index; i < jumlahData - 1; i++) {
    id[i] = id[i + 1];
    nama[i] = nama[i + 1];
    kategori[i] = kategori[i + 1];
    jumlah[i] = jumlah[i + 1];
}

jumlahData--;
```

editData() adalah Menggunakan linearSearchID() untuk menemukan index, lalu mengubah nilai langsung:


```

int index = linearSearchID(key);

if (index != -1) {

    System.out.println("-----");
    System.out.println("Data yang akan dihapus:");
    System.out.println("-----");
    System.out.println("ID      : " + id[index]);
    System.out.println("Nama    : " + nama[index]);
    System.out.println("Kategori : " + kategori[index]);
    System.out.println("Jumlah  : " + jumlah[index]);
}

```

3.3 Implementasi Searching

Linear Search (Nama & Kategori): Menelusuri seluruh array satu per satu dan membandingkan dengan kata kunci secara case-insensitive. Dapat menemukan lebih dari satu hasil.

```

for (int i = 0; i < jumlahData; i++) {
    if (nama[i].equalsIgnoreCase(key)) {
        System.out.println("Ditemukan: " + id[i] + " | " + nama[i] + " | " + kategori[i] + " | " + jumlah[i]);
        ditemukan = true;
    }
}

```

Binary Search (ID): Karena Binary Search mensyaratkan data terurut, dilakukan bubbleSortID() terlebih dahulu sebelum pencarian. Algoritma membagi rentang pencarian menjadi dua di setiap iterasi.

```
// Binary Search untuk mencari ID buku
static int binarySearch(int key) {
    int low = 0, high = jumlahData - 1;

    while (low <= high) {
        int mid = (low + high) / 2;
        if (id[mid] == key) return mid;
        else if (id[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

3.4 Implementasi Sorting

Bubble Sort (ID Ascending): Membandingkan dua elemen berdekatan dan menukar jika tidak urut. Setiap pass membawa elemen terbesar ke posisi akhir.

```
// Bubble Sort ascending berdasarkan ID buku
static void bubbleSortID() {
    for (int i = 0; i < jumlahData - 1; i++) {
        for (int j = 0; j < jumlahData - 1 - i; j++) {
            if (id[j] > id[j + 1]) {
                // tukar semua atribut
                swap(j, j + 1);
            }
        }
    }
}
```

Selection Sort (Nama: A-Z): Mencari elemen minimum dari sisa array yang belum terurut, lalu menukarnya ke posisi yang benar.

```
static void selectionSortNama() {
    for (int i = 0; i < jumlahData - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < jumlahData; j++) {
            if (nama[j].compareToIgnoreCase(nama[minIndex]) < 0) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            swap(i, minIndex);
        }
    }
}
```

Bubble Sort (Jumlah Descending): Sama dengan Bubble Sort ID, namun kondisi ditukar (<) agar urutan menurun.

```
// Bubble Sort descending berdasarkan jumlah buku
static void bubbleSortDesc() {
    for (int i = 0; i < jumlahData - 1; i++) {
        for (int j = 0; j < jumlahData - 1 - i; j++) {
            if (jumlah[j] < jumlah[j + 1]) {
                swap(j, j + 1);
            }
        }
    }
}
```

3.5 Fitur Tambahan

Statistik: Iterasi satu kali untuk menghitung total, max, min, dan rata-rata; kemudian iterasi per kategori untuk breakdown.

Simpan ke File: Menggunakan BufferedWriter untuk menulis data baris per baris dengan format ID|NAMA|KATEGORI|JUMLAH.

Load dari File: Menggunakan BufferedReader, melewati baris komentar (#), lalu mem-parse setiap baris dengan split("\\|").

BAB 4

ANALISIS KOMPLEKSITAS

4.1 Tabel Kompleksitas Waktu

No	Method/Operasi	Algoritma	Best Case	Average Case	Worst Case	Keterangan
1	linearSearchID(key)	Linear Search	$O(1)$	$O(n)$	$O(n)$	Berhenti di elemen pertama cocok
2	cariNama()	Linear Search	$O(1)$	$O(n)$	$O(n)$	Scan seluruh array
3	cariKategori()	Linear Search	$O(1)$	$O(n)$	$O(n)$	Semua kategori cocok dicetak
4	binarySearch(key)	Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	Data harus sudah terurut
5	cariID()	Sort + Binary Search	$O(n \log n)$	$O(n^2)$	$O(n^2)$	Didominasi bubbleSortID
6	bubbleSortID()	Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Best $O(n)$ jika sudah terurut
7	selectionSortNama()	Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Selalu dua loop penuh
8	bubbleSortDesc()	Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sama dengan bubbleSortID
9	tambahData()	Append + Linear Search	$O(n)$	$O(n)$	$O(n)$	Didominasi pengecekan duplikat

10	hapusData()	Shift Array	O(1)	O(n)	O(n)	Shift setelah elemen ditemukan
11	editData()	LinearSearch	O(1)	O(n)	O(n)	Didominasi pencarian ID
12	tampilData()	Iterasi	O(n)	O(n)	O(n)	Cetak semua elemen
13	tampilStatistik()	Iterasi	O(n)	O(n)	O(n)	Satu pass + loop kategori (4×n)
14	simpanData()	Iterasi + I/O	O(n)	O(n)	O(n)	Tulis n baris ke file
15	loadData()	Iterasi + I/O	O(n)	O(n)	O(n)	Baca n baris dari file
	swap(a, b)	Pertukaran	O(1)	O(1)	O(1)	Tukar 4 variabel, konstan

4.2 Kompleksitas Ruang (Space Complexity)

Komponen	Kompleksitas	Keterangan
Array id, nama, kategori, jumlah	O(n)	Kapasitas tetap 100, namun logis O(n) data
Variabel lokal sorting (temp)	O(1)	Hanya variabel swap sementara
Variabel lokal searching	O(1)	low, high, mid konstan
Keseluruhan program	O(n)	Didominasi penyimpanan array

4.3 Penjelasan Detail

Kompleksitas cariID() secara keseluruhan tetap $O(n^2)$

Meskipun Binary Search sendiri beroperasi pada $O(\log n)$, metode cariID() memanggil bubbleSortID() terlebih dahulu setiap kali dieksekusi. Bubble Sort memiliki kompleksitas $O(n^2)$ pada average dan worst case, sehingga kompleksitas total cariID() sepenuhnya didominasi oleh proses sorting-nya. Binary Search yang efisien di belakangnya tidak memberikan dampak signifikan terhadap performa keseluruhan. Solusi yang lebih optimal adalah menjaga data selalu terurut saat penambahan, atau menjalankan sort hanya sekali dengan menggunakan flag sudahTerurut.

Selection Sort selalu $O(n^2)$ tanpa pengecualian

Selection Sort menjalankan dua loop bersarang secara penuh di setiap kondisi baik data sudah terurut maupun acak karena tidak ada mekanisme early exit. Loop luar berjalan $n-1$ kali dan loop dalam selalu mencari minimum dari sisa elemen yang belum terurut. Hal ini berbeda dengan Bubble Sort yang secara teoritis dapat berhenti lebih awal apabila tidak ada pertukaran dalam satu pass, meskipun implementasi dalam program ini belum menggunakan optimasi flag tersebut.

Kompleksitas tampilStatistik() tetap linear $O(n)$

Meski terdapat beberapa loop di dalam method ini, seluruh loop bersifat sekuensial dan tidak ada yang bersarang satu sama lain. Loop pertama menghitung total stok, max, dan min dalam satu pass. Loop berikutnya mengiterasi 4 kategori, di mana setiap kategori menelusuri n data sehingga totalnya $4 \times n$ yang dalam notasi Big-O tetap disederhanakan menjadi $O(n)$ karena konstanta diabaikan.

BAB 5

PENGUJIAN

Karena laporan ini dibuat dalam format teks, berikut disajikan simulasi output terminal untuk setiap skenario pengujian.

5.1 Skenario 1 Tampil Data (Normal)

Input: Menu 1 → 2 (Tampilkan Data)

Output:

=====				
DAFTAR DATA BUKU				
=====				
ID	NAMA BUKU	KATEGORI	JUMLAH	

1	Matematika Dasar	Sains	15	
2	Fisika Modern	Sains	8	
3	Kimia Organik	Sains	12	
4	Biologi Sel	Sains	20	
5	Statistika Terapan	Sains	6	
6	Kalkulus Lanjut	Sains	9	
7	Pemrograman Java	Teknologi	25	
8	Basis Data	Teknologi	18	
9	Jaringan Komputer	Teknologi	11	
21	Sejarah Romawi	Sejarah	11	
22	Bahasa Indonesia	Bahasa	17	
23	Bahasa Inggris	Bahasa	23	
24	Bahasa Jepang	Bahasa	14	
25	Linguistik Umum	Bahasa	6	
26	Sastra Indonesia	Bahasa	10	
27	Tata Bahasa Inggris	Bahasa	19	
28	Bahasa Arab	Bahasa	8	
29	Pengantar Linguistik	Bahasa	12	
30	Semantik dan Pragmatik	Bahasa	5	
=====				
Total Data : 30 buku				

Hasil: Berhasil, seluruh 30 data ditampilkan dalam format tabel.

5.2 Skenario 2 Tambah Data (Berhasil)

Input: Menu 1 → 1, ID: 31, Nama: Kalkulus Vektor, Kategori: Sains, Jumlah: 10
Output:

```
=====
                        TAMBAH DATA BUKU
=====
Masukkan ID Buku       : 31
Masukkan Nama Buku     : Kalkulus Vektor
Masukkan Kategori      : Sains
Masukkan Jumlah Buku   : 10
=====
Data berhasil ditambahkan
=====
```

Hasil: Berhasil data ke-31 tersimpan, jumlahData menjadi 31.

5.3 Skenario 3 Tambah Data (ID Duplikat)

Input: Menu 1 → 1, ID: 1 (sudah ada)
Output:

```
=====
                        TAMBAH DATA BUKU
=====
Masukkan ID Buku       : 1
ID sudah digunakan
```

Hasil: Validasi berjalan, program menolak ID yang sama dan kembali ke menu.

5.4 Skenario 4 Tambah Data (Jumlah Negatif)

Input: Menu 1 → 1, ID: 99, Nama: Tes Buku, Kategori: Sains, Jumlah: -5

Output:

```
=====
TAMBAH DATA BUKU
=====
Masukkan ID Buku      : 99
Masukkan Nama Buku    : Tes Buku
Masukkan Kategori     : Sains
Masukkan Jumlah Buku  : -5
Jumlah buku tidak boleh negatif
```

Hasil: Validasi berjalan, program menolak nilai negatif.

5.5 Skenario 5 Edit Data

Input: Menu 1 → 3, ID: 1, Nama baru: Matematika Lanjut, Kategori baru: Sains, Jumlah baru: 20

Output:

```
Pilih menu: 3
Masukkan ID yang ingin diedit: 1
Nama baru: Matematika Lanjut
Kategori baru: Sains
Jumlah baru: 20
```

Hasil: Berhasil data ID 1 berubah, verifikasi dengan Tampil Data.

ID	NAMA BUKU	KATEGORI	JUMLAH
1	Matematika Lanjut	Sains	20

5.6 Skenario Hapus Data

Input: Menu 1 → 4, ID: 30

Output:

```

=====
                        HAPUS DATA BUKU
=====
Masukkan ID Buku : 30
-----
Data yang akan dihapus:
-----
ID          : 30
Nama        : Semantik dan Pragmatik
Kategori    : Bahasa
Jumlah      : 5
=====
Data berhasil dihapus
=====

```

Hasil: Berhasil — jumlahData berkurang menjadi 29.

26	Sastra Indonesia	Bahasa	10
27	Tata Bahasa Inggris	Bahasa	19
28	Bahasa Arab	Bahasa	8
29	Pengantar Linguistik	Bahasa	12

```

=====
Total Data : 29 buku

```

5.7 Skenario 7 Hapus Data (ID Tidak Ada)

Input: Menu 1 → 4, ID: 999

Output:

```

=====
                        HAPUS DATA BUKU
=====
Masukkan ID Buku : 999
Data tidak ditemukan

```

Hasil: Boundary handling bekerja.

5.8 Skenario 8 Cari Nama (Linear Search)

Input: Menu 2 → 1, Nama: Pemrograman Java

```
===== MENU SEARCHING =====
1. Cari Nama Buku
2. Cari ID Buku
3. Cari Kategori
0. Kembali
=====
Pilih menu: 1
Masukkan nama: Pemrograman Java
Ditemukan: 7 | Pemrograman Java | Teknologi | 25
```

Hasil: Berhasil ditemukan.

5.9 Skenario 9 Cari Nama (Tidak Ditemukan)

Input: Menu 2 → 1, Nama: Astronomi

Output:

```
===== MENU SEARCHING =====
1. Cari Nama Buku
2. Cari ID Buku
3. Cari Kategori
0. Kembali
=====
Pilih menu: 1
Masukkan nama: Astronomi
Data tidak ditemukan
```

Hasil: Berhasil, pesan tidak ditemukan muncul dengan benar.

5.10 Skenario 10 Cari ID (Binary Search)

Input: Menu 2 → 2, ID: 10

Output:

```
===== MENU SEARCHING =====
1. Cari Nama Buku
2. Cari ID Buku
3. Cari Kategori
0. Kembali
=====
Pilih menu: 2
Masukkan ID: 10
Ditemukan: 10 | Kecerdasan Buatan | Teknologi | 30
```

Hasil: Binary Search berhasil menemukan data setelah pre-sort.

5.11 Skenario 11 Cari Kategori

Input: Menu 2 → 3, Kategori: Sejarah

Output:

```
===== MENU SEARCHING =====
1. Cari Nama Buku
2. Cari ID Buku
3. Cari Kategori
0. Kembali
=====
Pilih menu: 3
Masukkan kategori: Sejarah
14 | Sejarah Indonesia | Sejarah | 10
15 | Sejarah Dunia | Sejarah | 13
16 | Perang Dunia II | Sejarah | 8
17 | Revolusi Industri | Sejarah | 5
18 | Sejarah Islam | Sejarah | 16
19 | Dinasti Ming | Sejarah | 4
20 | Kolonialisme Eropa | Sejarah | 9
21 | Sejarah Romawi | Sejarah | 11
```

Hasil: Berhasil, Semua 8 buku kategori Sejarah ditampilkan.

5.12 Skenario 12 Sort ID (Ascending)

Input: Menu 3 → 1

```
=====
Pilih menu: 1
=== Data Setelah Diurutkan ===
1 | Matematika Lanjut | Sains | 20
2 | Fisika Modern | Sains | 8
3 | Kimia Organik | Sains | 12
4 | Biologi Sel | Sains | 20
5 | Statistika Terapan | Sains | 6
6 | Kalkulus Lanjut | Sains | 9
7 | Pemrograman Java | Teknologi | 25
8 | Basis Data | Teknologi | 18
9 | Jaringan Komputer | Teknologi | 11

24 | Bahasa Jepang | Bahasa | 14
25 | Linguistik Umum | Bahasa | 6
26 | Sastra Indonesia | Bahasa | 10
27 | Tata Bahasa Inggris | Bahasa | 19
28 | Bahasa Arab | Bahasa | 8
29 | Pengantar Linguistik | Bahasa | 12
30 | Semantik dan Pragmatik | Bahasa | 5
```

Hasil: Data terurut ascending berdasarkan ID.

5.13 Skenario 13 Sort Nama (A-Z)

Input: Menu 3 → 2

Output:

```
Pilih menu: 2
=== Data Setelah Diurutkan (A-Z) ===
13 | Algoritma Pemrograman | Teknologi | 22
28 | Bahasa Arab | Bahasa | 8
22 | Bahasa Indonesia | Bahasa | 17
```

Hasil: Nama buku terurut alfabetis.

5.14 Skenario 14 Sort Jumlah Descending

Input: Menu 3 → 3

Output:

```
Pilih menu: 3
=== Data Setelah Diurutkan (Descending) ===
10 | Kecerdasan Buatan | Teknologi | 30
7 | Pemrograman Java | Teknologi | 25
23 | Bahasa Inggris | Bahasa | 23
13 | Algoritma Pemrograman | Teknologi | 22
4 | Biologi Sel | Sains | 20
1 | Matematika Lanjut | Sains | 20
27 | Tata Bahasa Inggris | Bahasa | 19
```

Hasil: Stok terbanyak di atas, terkecil di bawah.

5.15 Skenario 15 Statistik Data

Input: Menu 4 → 1

Output:

```
=====
                STATISTIK DATA PERPUSTAKAAN
=====
Total Judul Buku      : 30 buku
Total Stok Seluruh    : 392 eksemplar
Rata-rata Stok        : 13.07 eksemplar/judul
Stok Terbanyak        : Kecerdasan Buatan (30)
Stok Tersedikit       : Dinasti Ming (4)
```

```

--- Statistik per Kategori ---
Kategori      | Judul   | Total Stok   | Rata-rata
-----
Sains          | 6       | 75           | 12.50
Teknologi      | 7       | 127          | 18.14
Sejarah        | 8       | 76           | 9.50
Bahasa         | 9       | 114          | 12.67

--- Ketersediaan Stok ---
Tersedia (stok > 0) : 30 judul
Habis   (stok = 0)  : 0 judul
=====

```

Hasil: Berhasil, Semua kalkulasi statistik tampil dengan benar.

5.16 Skenario 16 Simpan dan Load Data

Input Simpan: Menu 4 → 2

Output:

```

=== ADDITIONAL FEATURES ===
1. Statistik Data
2. Simpan Data ke File
3. Load Data dari File
4. Status Sistem
0. Kembali
=====
Pilih menu: 2
[OK] Data berhasil disimpan ke: data_perpus.txt
      Total 30 buku tersimpan.

```

Input Load: Menu 4 → 3, konfirmasi: 1

```
=== ADDITIONAL FEATURES ===
1. Statistik Data
2. Simpan Data ke File
3. Load Data dari File
4. Status Sistem
0. Kembali
=====
Pilih menu: 3
[PERINGATAN] Load data akan menggantikan data yang ada saat ini!
Lanjutkan? (1=Ya / 0=Tidak): 1
[OK] Data berhasil dimuat dari: data_perpus.txt
      Total 30 buku berhasil dimuat.
```

Hasil: Data berhasil ditulis dan dibaca kembali dari file.

5.17 Skenario 17 Status Sistem

Input: Menu 4 → 4

Output:

```
Pilih menu: 4

-----
STATUS SISTEM PERPUSTAKAAN
-----

Status Terakhir : Data dimuat dari data_perpus.txt (30 buku)
Total Data      : 30 buku
[+] Tambah      : 0 kali
[~] Edit        : 0 kali
[-] Hapus       : 0 kali
-----
```

Catatan: Counter tambah/edit/hapus menunjukkan 0 karena terdapat bug yang belum diperbaiki (counter tidak di-increment dalam method yang bersangkutan).

BAB 6

KESIMPULAN

6.1 Ringkasan

Program SistemPerpus berhasil dibangun sebagai sistem manajemen perpustakaan digital berbasis konsol Java. Sistem ini mencakup empat modul utama yaitu CRUD, Searching, Sorting, dan Additional Features yang masing-masing telah diimplementasikan dan diuji.

Algoritma yang digunakan meliputi:

- **Linear Search** untuk pencarian nama dan kategori dengan kompleksitas $O(n)$
- **Binary Search** untuk pencarian ID dengan kompleksitas $O(\log n)$ meskipun secara total $O(n^2)$ karena pre-sort
- **Bubble Sort** untuk pengurutan ID dan jumlah dengan kompleksitas $O(n^2)$
- **Selection Sort** untuk pengurutan nama dengan kompleksitas $O(n^2)$

6.2 Kelebihan Sistem

1. Boundary handling sudah mencakup: kapasitas penuh, ID duplikat, jumlah negatif, dan data kosong.
2. Tampilan antarmuka konsol rapi menggunakan printf berformat.
3. Persistensi data berjalan baik dengan mekanisme simpan/load file teks.
4. Statistik data cukup informatif dengan breakdown per kategori.

6.3 Kelemahan dan Saran Perbaikan

Kelemahan	Saran
Counter tambah/edit/hapus tidak di-increment	Tambahkan counterTambah++ dll. setelah operasi berhasil

Input tidak aman terhadap tipe salah (crash jika huruf diinput saat diminta angka)	Gunakan try-catch InputMismatchException
Slot array kotor saat tambahData() gagal di tengah	Validasi semua input sebelum menyimpan ke array
statusTerakhir tidak konsisten diperbarui	Update di semua operasi yang berhasil
cariID() selalu sort ulang meski data sudah urut	Gunakan flag sudahTerurut
Typo nama method bubbleSortDesc	Ganti menjadi bubbleSortDesc
Struktur parallel array rawan inkonsistensi	Refactor menggunakan class Buku

6.4 Penutup

Proyek ini memberikan pemahaman praktis tentang penerapan struktur data array, algoritma searching dan sorting, serta manajemen file dalam Java. Meskipun terdapat beberapa kelemahan teknis, sistem secara keseluruhan berfungsi sesuai dengan kebutuhan yang telah dianalisis dan dapat dikembangkan lebih lanjut menggunakan pendekatan berorientasi objek.